

ADR-01 Uso de Azure Event Hubs como punto de entrada masivo	
Contexto	Actualmente, PayFlow opera sobre una arquitectura monolítica del 2020 que solo procesa 40 transacciones por segundo, lo que genera demoras de hasta 8 segundos en temporadas de alta demanda. Para alcanzar el objetivo de procesar 500 eventos por segundo con baja latencia, necesito un componente en Azure que actúe como punto de entrada asíncrono para desacoplar el sistema y evitar que los fallos en un paso bloqueen todo el proceso.
Alternativas	He evaluado como alternativas Azure Service Bus y Azure Storage Queues para el punto de entrada de las transacciones. Mientras que Service Bus ofrece una alta confiabilidad y Storage Queues destaca por su bajo costo, ambas presentan limitaciones frente a las 500 transacciones por segundo que requiere PayFlow, ya que no están optimizadas para el procesamiento masivo de ráfagas o el particionamiento de eventos en tiempo real.
Decisión	He decidido implementar Azure Event Hubs, ya que es el único servicio diseñado específicamente para actuar como un buffer de alto rendimiento capaz de absorber picos de demanda masivos sin bloquear el sistema.
Justificación	He seleccionado Azure Event Hubs porque está diseñado específicamente para la ingesta de streaming y el manejo de un alto volumen de eventos. Dado que el requerimiento técnico de PayFlow exige soportar picos de hasta 500 transacciones por segundo para eliminar el cuello de botella actual, este servicio actúa como el buffer ideal. Por otro lado, he decidido reservar Azure Service Bus exclusivamente para la mensajería empresarial y las transacciones de alto valor superiores a los \$5.000.000 COP, donde la prioridad es el control individual del mensaje y no el volumen masivo de entrada.
Consecuencia	Como beneficio principal, logro una alta escalabilidad y el desacoplamiento total de la entrada del sistema, lo que me permite procesar picos masivos de hasta 500 tx/s con baja latencia. El principal trade-off es que debo gestionar la complejidad de las particiones y aceptar una retención de mensajes limitada a un día en el nivel básico. Además, asumo el reto técnico de asegurar que el procesamiento posterior sea capaz de manejar posibles duplicados para mantener la integridad de los datos de PayFlow.

ADR-02: Implementación de lógica de negocio con Azure Functions	
Contexto	Una vez que los eventos ingresan al sistema, necesito un componente capaz de procesar la lógica de negocio en tiempo real. El sistema actual de PayFlow falla al detectar el fraude de manera inmediata y no escala ante los picos de demanda. Requiero una solución que me permita aplicar validaciones personalizadas para cumplir con la regulación de transacciones de alto valor, manteniendo costos operativos bajos mediante un modelo de pago por uso y garantizando la observabilidad total de cada operación para mitigar errores.
Alternativas	He comparado Azure Stream Analytics frente a Azure Functions como motores de procesamiento. Aunque Stream Analytics es una opción robusta para el análisis estadístico en tiempo real mediante consultas SQL, presenta una rigidez que me dificultaría implementar las reglas de validación de datos y la lógica de detección de fraude personalizada que PayFlow necesita.
Decisión	Decidí elegir Azure Functions por su flexibilidad para ejecutar código personalizado (Python), lo que me permite manipular cada JSON individualmente y escalar de forma automática bajo un modelo de pago por consumo, garantizando así un procesamiento eficiente, de bajo costo y totalmente alineado con los requerimientos de observabilidad y regulación del caso.
Justificación	Cuento con experiencia sólida en Python y Node.js para desarrollar la lógica de PayFlow. Este servicio me permite implementar, mediante código personalizado en la nube de Azure, todas las validaciones de datos, reglas antifraude, enrutamiento y la escritura directa en la base de datos, garantizando un control total sobre el procesamiento de cada transacción de manera eficiente y escalable.
Consecuencia	Al adoptar esta solución, obtengo el beneficio de un escalamiento automático que se ajusta a los picos de demanda de PayFlow y un ahorro significativo al pagar solo por el tiempo de ejecución. Como principal trade-off, asumo la responsabilidad de gestionar y mantener el código fuente de las funciones, además de considerar una posible latencia mínima (cold start) en la primera ejecución tras periodos de inactividad, la cual mitigaremos mediante una lógica de procesamiento eficiente.

ADR-03: Persistencia de transacciones en Azure Cosmos DB	
Contexto	Necesito almacenar el estado final de cada transacción procesada por PayFlow para auditoría y consulta. El sistema actual tiene esquemas rígidos que dificultan la evolución del modelo de datos. Para cumplir con los requerimientos de baja latencia y alta disponibilidad, busco una base de datos que soporte el crecimiento masivo de datos sin comprometer la velocidad de respuesta ante picos de demanda.
Alternativas	Para la persistencia, comparé Azure SQL Database frente a Azure Cosmos DB. Aunque Azure SQL es excelente para la integridad relacional, su esquema rígido complica el manejo de eventos en formato JSON que pueden variar con el tiempo. Por el contrario, elegí Azure Cosmos DB porque, al ser una base de datos NoSQL orientada a documentos, me ofrece la flexibilidad necesaria para almacenar esquemas dinámicos y garantiza una latencia de milisegundos a escala global, lo cual es vital para el volumen de transacciones de una fintech.
Decisión	He decidido implementar Azure Cosmos DB utilizando la API de NoSQL. Esta elección me permite registrar cada transacción como un documento JSON de forma inmediata, facilitando la escalabilidad horizontal y asegurando que los datos estén disponibles en tiempo real tanto para el negocio como para los sistemas de monitoreo de fraude.
Justificación	Se decidió por Azure Cosmos DB porque se adapta perfectamente a la naturaleza de los eventos en formato JSON de PayFlow y permite realizar escrituras extremadamente rápidas, lo cual es crítico para no generar retrasos en el procesamiento masivo de datos. Su modelo NoSQL me brinda la flexibilidad necesaria para evolucionar el esquema de las transacciones sin afectar la disponibilidad del sistema, garantizando siempre la baja latencia que requiere una fintech de este nivel para sus operaciones en tiempo real.
Consecuencia	Como beneficio principal, obtengo una base de datos extremadamente rápida y flexible que escala automáticamente. El principal trade-off es que debo ser muy cuidadoso con la gestión de las Unidades de Rendimiento (RUs) para evitar costos elevados, por lo que configuraré alertas de presupuesto y utilizaré el nivel gratuito de Azure para el desarrollo inicial del proyecto.

ADR-04: Orquestación de transacciones de alto valor con Azure Service Bus	
Contexto	Para las transacciones de alto valor (mayores a \$5.000.000 COP), PayFlow requiere una garantía de entrega absoluta y procesos de auditoría más estrictos. El sistema actual no diferencia estas operaciones críticas del flujo común, lo que aumenta el riesgo de pérdida de datos durante picos de demanda. Necesito un servicio de mensajería empresarial que soporte flujos de trabajo complejos, reintentos automáticos y que asegure que ningún mensaje se procese a medias.
Alternativas	Se evaluó el uso de Azure Storage Queues frente a Azure Service Bus. Aunque Storage Queues es una opción muy económica y sencilla para colas de gran volumen, carece de funciones avanzadas como el manejo de sesiones, la detección de duplicados y la garantía de "entrega exacta" (at-least-once o at-most-once) de forma nativa. Por el contrario, he elegido Azure Service Bus porque, aunque tiene un costo operativo ligeramente mayor, me ofrece las capacidades de mensajería empresarial necesarias para tratar estas transacciones como críticas, permitiendo un control total sobre el estado de cada operación.
Decisión	Se decidió implementar Azure Service Bus para el canal de transacciones de alto valor. Esta decisión me permite separar el tráfico crítico del flujo general de Event Hubs, asegurando que estas operaciones cuenten con una cola dedicada que soporte transacciones atómicas y un manejo de errores (Dead-lettering) mucho más sofisticado.
Justificación	Se seleccionó Azure Service Bus porque las transacciones de alto valor en PayFlow no pueden tratarse simplemente como eventos masivos, sino como mensajes que requieren un compromiso de entrega total. Las operaciones mayores a \$5.000.000 COP exigen mayor confiabilidad, procesos de reintento automáticos, auditoría y un control que una cola simple no puede proporcionar. La capacidad de este servicio para manejar sesiones y garantizar el orden (FIFO) me permite asegurar que el procesamiento de grandes sumas de dinero sea seguro y cumpla con todas las expectativas de regulación del negocio.
Consecuencia	El principal beneficio es la máxima fiabilidad y el control detallado sobre las transacciones financieras más importantes del sistema. El trade-off que asumo es una mayor complejidad en la configuración del servicio y un costo superior comparado con las colas básicas, pero es un costo justificado para mitigar el riesgo financiero que representan estos movimientos de capital.

ADR-05: Observabilidad y monitoreo mediante Azure Monitor y Application Insights	
Contexto	Con la nueva arquitectura distribuida (Event Hubs, Functions, Cosmos DB), PayFlow necesita una visibilidad clara de punta a punta para detectar fallos rápidamente. El sistema anterior carecía de métricas en tiempo real, lo que dificultaba identificar por qué una transacción fallaba o dónde se generaban los retrasos. Necesito una solución que me permita rastrear cada mensaje, monitorear la salud de los servicios y recibir alertas automáticas ante anomalías en el sistema.
Alternativas	Logramos evaluar el uso de herramientas de terceros como Datadog o New Relic frente a la suite nativa de Azure Monitor + Application Insights. Aunque las herramientas externas ofrecen tableros muy avanzados y capacidades multicloud, implican una configuración adicional de agentes y un costo de transferencia de datos fuera de la red de Azure. Por el contrario, he elegido la solución nativa de Azure porque ofrece una integración inmediata ("un solo clic") con las Functions y el Event Hub, permitiéndome ver trazas completas sin salir del portal y manteniendo los costos controlados dentro del mismo presupuesto.
Decisión	Se decidió implementar Azure Monitor junto con Application Insights como mi plataforma central de observabilidad. Esta combinación me permite recolectar logs, métricas y trazas distribuidas de todos los componentes de PayFlow en un solo lugar, facilitando la creación de tableros de control personalizados y el rastreo de transacciones individuales desde que entran al sistema hasta que se guardan en la base de datos.
Justificación	Se logro seleccionar estas herramientas porque la integración nativa con Azure Functions y Cosmos DB me permite obtener métricas de rendimiento y errores de forma automática sin escribir código adicional. Esto es vital para cumplir con los requerimientos de observabilidad del caso, ya que puedo configurar alertas inmediatas si el tiempo de respuesta supera los límites permitidos o si se detectan patrones de error que sugieran un intento de fraude. Al estar todo dentro de Azure, la latencia en la recolección de datos es mínima y la seguridad de los logs está garantizada por el mismo control de acceso (RBAC) del proyecto.
Consecuencia	El beneficio principal es tener una visibilidad total y unificada que acelera la resolución de problemas (MTTR). Como trade-off, debo ser cuidadoso con el volumen de datos de telemetría enviados para evitar sobrecostos por ingesta de logs, por lo que aplicaré políticas de muestreo (sampling) para registrar el 100% de los errores pero solo un porcentaje representativo de las transacciones exitosas.

